25 *FastAPI* Interview Questions

The essential questions tested in modern GenAI backend and ML engineering interviews.

#FastAPI #GenAI #Architecture

Naresh Edagotti · Follow · PracticAI

Repost

FUNDAMENTALS

Core *Concepts*

What is FastAPI, and why use it for AI?

FastAPI is a modern framework for APIs. It's preferred for ML because of great async support and automatic validation. Flask is slower; Django is too heavy.

How does it achieve high performance?

It leverages ASGI for async operations, Starlette for speed, and Pydantic for validation. It ranks alongside Node.js or Go frameworks in benchmarks.

What is ASGI vs WSGI?

ASGI supports async execution, handling multiple requests concurrently without blocking. WSGI is synchronous, forcing the server to block during simple I/O calls.

Check caption for AI Interview Kit

MASTER CURRICULUM

AI Interview *Kit*

After seeing how AI interviews are evolving, I created a single kit covering the areas companies **actually test**.

- AI Fundamentals
- Transformers
- LLM Architecture
- Prompt Engineering
- Fine-tuning
- RAG Systems
- AI Agents
- Memory & Vector DB
- AI Backend systems
- Deployment strategies
- GenAI System Design

Everything structured to help you prepare.

Check caption for AI Interview Kit

ARCHITECTURE

Architecture & *Design*

Role of Uvicorn vs Starlette?

Uvicorn is the lightning-fast ASGI server that runs the app. Starlette is the lightweight web framework FastAPI uses internally for routing and requests.

How does Auto API Docs work?

FastAPI automatically serves interactive Swagger UI at /docs. It implicitly generates this from your Python code, models, and type hints instantly.

What do route decorators do?

Decorators like @app.post register your functions as handlers. They define exact mappings so requests automatically route to the right Python function.

Check caption for AI Interview Kit

PYDANTIC

Validation & *Data*

How are requests serialized?

FastAPI uses Pydantic to instantly parse incoming JSON into validated Python objects, and serialize Python objects cleanly back into JSON responses.

What is Pydantic?

A powerful data validation library driven by Python type hints. FastAPI uses it to enforce schemas, format errors, and self-document the API.

How does Auto Validation work?

It cross-references incoming payloads with your Pydantic schemas. If wrong, it refuses the request and auto-emits a 422 error detailing the exact issue.

Check caption for AI Interview Kit

DATA TYPES

Advanced *Types*

What is a BaseModel?

Pydantic's core class. You inherit from it (e.g., class Input(BaseModel):) to explicitly define strictly-typed request or response structures.

Why rely on Type Hints?

Type hints (e.g., prompt: str) eliminate manual type-checking boilerplate. They actively drive input validation, IDE autocompletion, and schema generation.

Handling Invalid Client Data?

FastAPI short-circuits execution and returns a 422 Unprocessable Entity error indicating the broken field, protecting your main code from malformed data.

Check caption for AI Interview Kit

ASYNCHRONOUS

Concurrency & *Async*

Optionals & Constraints

Use typing.Optional for non-compulsory inputs. Apply Pydantic's Field(gt=0, max_length=100) to strictly bound numerical ranges and string lengths.

Request vs Response Models

Decouple inputs from outputs. Request models process inbound data, while Response models enforce safe outbound schemas (e.g. omitting passwords).

What does async def mean?

It designates a coroutine. It tells the server it can pause execution of this function during I/O waits, freeing resources to instantly serve other incoming users.

Check caption for AI Interview Kit

PERFORMANCE

Performance & *Scale*

async def vs normal def

Use async def strictly for non-blocking I/O (DB calls, external API fetches). Use normal def for heavy CPU math, otherwise you'll freeze the entire async loop.

Concurrency in FastAPI

Handling multiple requests on a single thread. It seamlessly interleaves waiting periods. If 50 users query a slow DB, FastAPI processes them all collectively.

Managing I/O-bound Tasks

Handled flawlessly via async/await mappings. The moment a DB query initiates, FastAPI pauses the current route and instantaneously jumps to handle another request.

Check caption for AI Interview Kit

BEST PRACTICES

GenAI *Design* Limits

Can it run CPU-heavy ML inference?

Heavy ML natively blocks the event loop. Solution: Execute via background tasks, separate ProcessPoolExecutors, or dedicated API inference worker pools.

Scenario: RAG API Structure

Define an async POST route. Pydantic validates query. Await vector DB context retrieval, await generated LLM response, and return the bundled output.

Scenario: Preventing DB Blocks

Always utilize async-compatible clients (e.g., motor, asyncpg, httpx). Synchronous drivers force the entire server to halt until they complete.

Check caption for AI Interview Kit

SYSTEMS

Scenario Based *Tricks*

Scenario: High Payload Limits

Cap lengths firmly in Pydantic. Use middleware to drop abnormally large JSON bodies before parsing, preserving server memory and mitigating abuse.

Scenario: Slow 10s Responses

Never leave client connections hanging. Return an immediate 202 status with a Task ID, dispatching the slow ML task to Celery or FastAPI BackgroundTasks.

Scenario: Validating Constraints

Bind Pydantic Field restrictions to inputs: Field(ge=0, le=1.0) guarantees temperature constraints without writing a single line of internal validation logic.

Check caption for AI Interview Kit

DEBUGGING

Real-World *Debugging*

Scenario: Crashing under Load

Investigate connection pool exhaustion first—async connects quickly scale. Profile memory, integrate strict rate limits, and make sure CPU tasks aren't blocking.

Check caption for AI Interview Kit